

EIN092B: Laboratorio de Interfaz gráfica para el procesamiento de imágenes

Guía de Actividades Sesión 14

10 al 14 de noviembre de 2025

1. Requisitos de entrada.

- Haber **entendido** y **realizado** las actividades de todas las sesiones anteriores.

2. Objetivos

- Comprender la arquitectura de aplicaciones de escritorio en Python utilizando la biblioteca estándar **Tkinter**.
- Diseñar e implementar una interfaz gráfica de usuario (GUI) nativa para la carga y manipulación interactiva de imágenes digitales.
- Aplicar filtros convolucionales (kernels) de suavizado y detección de bordes, conectando la lógica de *widgets* (botones, menús) a funciones de **OpenCV**.
- Implementar el Análisis de Componentes Principales (PCA) para la reconstrucción de imágenes, utilizando un *widget* de *slider* para la selección dinámica de componentes.

3. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio.

3.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.
- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.

- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que **TODOS** los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la Sección 3.2.
- En el caso de las actividades no entregadas en la sección correspondiente, contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso (penalización correspondiente al 35 % del puntaje inicial si se entrega en la siguiente sesión. Si se entrega en dos sesiones posteriores, 50 %).**
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

3.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre los códigos propuestos por el grupo. La segunda parte constará de una verificación de sus códigos y respuestas, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se le indican a continuación:

1. Debe crear una carpeta con el nombre *GXX_Sesion14*, donde *XX* el número de grupo. Por ejemplo, la carpeta para el grupo 5 se llamaría *G5_Sesion14*. El número de su grupo fue definido en la sesión anterior.
2. Dentro de esta carpeta, debe incluirse un archivo de Jupyter Notebook que contenga el desarrollo de las actividades y las respuestas correspondientes.

Recordar que **será responsabilidad del grupo entregar de manera correcta los archivos**. Se permitirá el envío de los archivos hasta las **23:59 del 15 de Noviembre**. Sin embargo, solo se evaluarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no será considerada en la revisión.

4. Introducción

En lugar de depender de aplicaciones web, podemos construir interfaces nativas de escritorio que se ejecutan directamente en el sistema operativo. La biblioteca estándar de Python para esto es **Tkinter**.

4.1. Tkinter: Interfaces Nativas en Python

Tkinter ("Tk interface") es el kit de herramientas de GUI estándar de Python. Es una envoltura ligera alrededor de Tcl/Tk, un lenguaje de scripting para GUIs. Al estar en la biblioteca estándar, no requiere instalación.

Conceptos Clave de Tkinter:

1. **Ventana Raíz (Root Window):** Es la ventana principal de la aplicación, creada con `tk.Tk()`.
2. **Widgets:** Son los componentes de la GUI (botones, etiquetas, sliders, etc.). Ejemplos: `tk.Button`, `tk.Label`, `tk.Scale`.
3. **Gestores de Geometría:** Son los métodos que organizan los *widgets* en la ventana. Los más comunes son `.pack()` (apila widgets) y `.grid()` (los organiza en una grilla).
4. **Manejo de Eventos:** Se conecta la interacción del usuario (ej. clic de botón) a una función de Python usando el argumento `command`.
5. **Loop Principal (Main Loop):** `root.mainloop()` es una llamada bloqueante que inicia el bucle de eventos de la aplicación, esperando la interacción del usuario.

```
1 import tkinter as tk
2
3 # 1. Crear la ventana raiz
4 root = tk.Tk()
5 root.title("Mi Primera App")
6
7 # 2. Crear un widget
8 # tk.Label es una etiqueta de texto
9 label = tk.Label(root, text="Hola, mundo!")
10
11 # 3. Empaquetar el widget
12 # .pack() lo coloca en la ventana
13 label.pack(pady=10, padx=10)
14
15 # 4. Iniciar el bucle de eventos
16 root.mainloop()
```

Tkinter y OpenCV/NumPy: Tkinter no puede mostrar una imagen de NumPy (OpenCV) directamente. Necesitamos una biblioteca intermediaria: **Pillow** (PIL). El flujo es: **NumPy (OpenCV) → PIL Image → ImageTk.PhotoImage → tk.Label**

4.2. Análisis de Componentes Principales (PCA) en Imágenes

El **Análisis de Componentes Principales (PCA)** es una técnica de reducción de dimensionalidad. Su objetivo es transformar un conjunto de datos con variables posiblemente correlacionadas en un nuevo conjunto de variables linealmente no correlacionadas llamadas **componentes principales**.

Estas componentes se ordenan según la cantidad de **varianza** original que “explican”. La primera componente principal captura la mayor cantidad de varianza, la segunda la siguiente, y así sucesivamente.

PCA para Reconstrucción de Imágenes: Una imagen en escala de grises de $N \times M$ píxeles puede ser tratada como una matriz. Al aplicar PCA, podemos “comprimir” la imagen reteniendo solo las primeras k componentes principales.

La reconstrucción de la imagen se realiza utilizando la transformación inversa de PCA, pero usando solo el subconjunto k de componentes.

$$X_{reconstruido} = \text{Transformada}_k \cdot \text{Componentes}_k^T + \mu,$$

donde μ es la media de los datos originales.

Implementación con Scikit-Learn: El código de `scikit-learn` para calcular el PCA es independiente de la GUI y se puede reutilizar.

```

1 import numpy as np
2 from sklearn.decomposition import PCA
3
4 def aplicar_pca(imagen_np, k):
5     # 1. Normalizar (Centrar la media)
6     img_norm = imagen_np - np.mean(imagen_np, axis=0)
7
8     # 2. Aplicar PCA
9     pca = PCA(n_components=k)
10    transformed = pca.fit_transform(img_norm)
11
12    # 3. Reconstruir
13    reconstructed = pca.inverse_transform(transformed)
14
15    # 4. Devolver a la escala original (sumar la media)
16    reconstructed_img = reconstructed + np.mean(imagen_np, axis=0)
17
18    # 5. Asegurar que los valores estan en el rango de pixeles
19    reconstructed_img = np.clip(reconstructed_img, 0, 255).astype(np.uint8)
20
21    return reconstructed_img

```

5. Actividades

El objetivo es construir una aplicación en una sola clase de Python. Usaremos una estructura Orientada a Objetos, que es la forma estándar de construir aplicaciones en Tkinter.

Estructura Base (Esqueleto de la App): Copie este código como su punto de partida.

```

1 import tkinter as tk
2 from tkinter import filedialog
3 from PIL import Image, ImageTk

```

```
4 import cv2
5 import numpy as np
6 from sklearn.decomposition import PCA
7
8 class ImageProcessorApp:
9     def __init__(self, root):
10         self.root = root
11         self.root.title("Procesador de Imágenes")
12
13         # Guardar referencias a las imagenes
14         self.original_image_np = None # Imagen en formato NumPy
15         self.processed_image_tk = None # Imagen en formato Tkinter
16
17         # --- Configurar la GUI ---
18
19         # 1. Frame de controles
20         control_frame = tk.Frame(self.root)
21         control_frame.pack(side=tk.TOP, fill=tk.X, padx=10, pady=5)
22
23         self.load_button = tk.Button(control_frame, text="Cargar Imagen", command=
24 self.cargar_imagen)
25         self.load_button.pack(side=tk.LEFT)
26
27         # 2. Frame de visualizacion (dividido en dos)
28         display_frame = tk.Frame(self.root)
29         display_frame.pack(fill=tk.BOTH, expand=True)
30
31         self.original_label = tk.Label(display_frame, text="Imagen Original")
32         self.original_label.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=5)
33
34         self.processed_label = tk.Label(display_frame, text="Imagen Procesada")
35         self.processed_label.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx
36 =5)
37
38     def cargar_imagen(self):
39         # A implementar en Actividad 1
40         pass
41
42     def aplicar_filtro(self):
43         # A implementar en Actividad 2
44         pass
45
46     def reconstruir_pca(self):
47         # A implementar en Actividad 3
48         pass
49
50 # --- Funcion para convertir de NumPy (OpenCV) a Tkinter Image ---
51 def np_to_tkimage(np_image):
52     # OpenCV usa BGR, PIL usa RGB (si es color)
53     # Para escala de grises, es mas simple
54     if len(np_image.shape) == 2: # Escala de grises
55         img_pil = Image.fromarray(np_image)
56     else: # Color
57         img_pil = Image.fromarray(cv2.cvtColor(np_image, cv2.COLOR_BGR2RGB))
58
59     return ImageTk.PhotoImage(image=img_pil)
```

```

59
60 if __name__ == "__main__":
61     root = tk.Tk()
62     root.geometry("800x600") # Tamano inicial
63     app = ImageProcessorApp(root)
64     root.mainloop()

```

5.1. Actividad 1: Carga y Visualización de Imagen

Complete el método `cargar_imagen`.

Lineamientos:

- Use `filedialog.askopenfilename()` para mostrar un diálogo de selección de archivo.
- Lea la imagen usando `cv2.imread()` en modo escala de grises (`cv2.IMREAD_GRAYSCALE`).
- Almacene esta imagen de NumPy en `self.original_image_np`.
- Convierta la imagen de NumPy a una imagen de Tkinter usando la función `np_to_tkimage` proporcionada.
- **Importante:** Guarde la referencia de la imagen de Tkinter en un atributo de la clase (ej. `self.original_image_tk`). Si la guarda en una variable local, Python la borrará (garbage collector) y la imagen no se mostrará.
- Configure la `self.original_label` para que muestre esta imagen.

```

1  def cargar_imagen(self):
2      # 1. Pedir al usuario el path del archivo
3      file_path = filedialog.askopenfilename()
4      if not file_path:
5          return # El usuario cancelo
6
7      # 2. Leer la imagen con OpenCV en escala de grises
8      self.original_image_np = cv2.imread(file_path, cv2.IMREAD_GRAYSCALE)
9
10     # 3. Convertir a formato Tkinter
11     tk_image = np_to_tkimage(self.original_image_np)
12
13     # 4. Mostrar en la etiqueta original
14     self.original_label.config(image=tk_image, text="")
15
16     # 5. GUARDAR LA REFERENCIA (CRUCIAL)
17     self.original_image_tk = tk_image
18
19     # 6. Tambien mostrarla en el panel de procesado al inicio
20     self.processed_label.config(image=tk_image, text="")
21     self.processed_image_tk = tk_image

```

5.2. Actividad 2: Aplicación de Kernels Interactivos

Agregue un menú de selección (dropdown) y un botón para aplicar filtros.

Lineamientos:

- Modifique el `__init__` para añadir un `tk.StringVar` y un `tk.OptionMenu` al `control_frame`.
- Las opciones deben ser: 'Original', 'Suavizado (Blur)', 'Bordes (Laplace)'.
- Añada un `tk.Button` "Aplicar Filtro" que llame a `self.aplicar_filtro`.
- Complete el método `aplicar_filtro`:
- Verifique que se haya cargado una imagen (`self.original_image_np is not None`).
- Obtenga la selección del `tk.StringVar`.
- Aplique el kernel correspondiente usando `cv2.filter2D`.
- Muestre la imagen resultante en `self.processed_label` (recordando el flujo NumPy → PIL → ImageTk y guardando la referencia).

```
1      # ... (despues del boton de cargar)
2
3      # Menu de seleccion de filtro
4      self.filter_options = ['Original', 'Suavizado (Blur)', 'Bordes (Laplace)']
5
6      self.filter_var = tk.StringVar(self.root)
7      self.filter_var.set(self.filter_options[0]) # Valor por defecto
8
9      filter_menu = tk.OptionMenu(control_frame, self.filter_var, *self.filter_options)
10     filter_menu.pack(side=tk.LEFT, padx=5)
11
12     apply_filter_btn = tk.Button(control_frame, text="Aplicar Filtro",
13                                command=self.aplicar_filtro)
14     apply_filter_btn.pack(side=tk.LEFT)
```

```
1     def aplicar_filtro(self):
2         if self.original_image_np is None:
3             return # No hay imagen cargada
4
5         filtro = self.filter_var.get()
6         processed_np = None
7
8         if filtro == 'Original':
9             processed_np = self.original_image_np
10
```

```

11         elif filtro == 'Suavizado (Blur)':
12             kernel_blur = np.ones((5, 5), np.float32) / 25.0
13             processed_np = cv2.filter2D(self.original_image_np, -1, kernel_blur)
14
15         elif filtro == 'Bordes (Laplace)':
16             # SU CODIGO AQUI: Defina el kernel Laplaciano (ej. 3x3)
17             kernel_laplace = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], np.
float32)
18             # SU CODIGO AQUI: Aplique el filtro
19             processed_np = cv2.filter2D(self.original_image_np, -1,
kernel_laplace)
20
21             # Mostrar la imagen procesada
22             tk_image = np_to_tkimage(processed_np)
23             self.processed_label.config(image=tk_image)
24             self.processed_image_tk = tk_image # Guardar referencia

```

5.3. Actividad 3: PCA y Reconstrucción Interactiva

Agregue un *slider* (llamado `tk.Scale`) para controlar el número de componentes de PCA.

Lineamientos:

- Modifique el `__init__` para añadir un `tk.Scale` y un botón Reconstruir PCA”.
- El `tk.Scale` debe ser horizontal (`orient=tk.HORIZONTAL`).
- El rango del `tk.Scale` es un desafío: no sabemos el tamaño de la imagen al inicio. Configúrelo de 1 a 100 por ahora. Lo ideal es actualizar su rango (opción `to=...`) después de cargar la imagen.
- Complete el método `reconstruir_pca`:
- Obtenga el valor del slider (`k = self.pca_slider.get()`).
- Llame a la función `aplicar_pca` (definida en la Introducción, debe copiarla en su script) con `self.original_image_np` y `k`.
- Muestre la imagen reconstruida en `self.processed_label`.

```

1         # ... (dentro de control_frame)
2
3         # Slider para PCA
4         self.pca_slider = tk.Scale(control_frame, from_=1, to=100, orient=tk.
HORIZONTAL, label="Componentes PCA")
5         self.pca_slider.pack(side=tk.LEFT, padx=10)
6

```



```
7     apply_pca_btn = tk.Button(control_frame, text="Reconstruir PCA", command=
    self.reconstruir_pca)
8     apply_pca_btn.pack(side=tk.LEFT)
```

```
1     def reconstruir_pca(self):
2         if self.original_image_np is None:
3             return
4
5         # Obtener k del slider
6         k = int(self.pca_slider.get())
7
8         # (Mejora opcional: actualizar el 'to' del slider en cargar_imagen)
9         # max_comp = min(self.original_image_np.shape)
10        # self.pca_slider.config(to=max_comp)
11
12        # SU CODIGO AQUI: Llame a la funcion de PCA
13        # (Asegurese de haber copiado la funcion aplicar_pca en su script)
14        reconstructed_np = aplicar_pca(self.original_image_np, k)
15
16        # Mostrar resultado
17        tk_image = np_to_tkimage(reconstructed_np)
18        self.processed_label.config(image=tk_image)
19        self.processed_image.tk = tk_image # Guardar referencia
```

Referencias

- [1] Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. <https://docs.python.org/3/library/tkinter.html>
- [2] Scikit-learn: Machine Learning in Python. *sklearn.decomposition.PCA*. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
- [3] OpenCV (2025). *OpenCV Python Tutorials*. https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html